SIH26122 — Extractor Data & Implementation Specification

Context + synthetic-data-first implementation brief for Claude Code

0. PROJECT CONTEXT — WHAT IS SIH26122?

Problem Statement: “Intelligent Data Capture & Schedule-Linking Layer for Infrastructure Project Management: Real-Time Actual Progress Tracking (Planning-to-Execution Bridge).” Organization: **Oil India Limited**. Category: **Software**. Theme: **Smart Automation**.

Infrastructure projects have a structured baseline plan. Work is broken down from high-level milestones into increasingly detailed WBS levels, ultimately reaching executable **L5/L6 activities**. Baseline schedules are maintained in tools such as Primavera or MS Project.

The problem is that actual field execution does not arrive in the same structured form. Supervisors and site teams may report progress through daily progress reports, site diaries, discipline spreadsheets, or verbal updates. These reports can use different wording, abbreviations, formats and levels of detail, and they often do not contain the L5/L6 activity ID used by the schedule.

Example: The schedule may contain “Erect 24in Line 24-XX”, while a supervisor writes “24 inch spool erection done at Rack B from 10 to 4.” A human understands the relationship, but software needs an intelligent layer to extract the event and determine which planned activity it belongs to.

Our goal: build an AI-powered bridge between messy field execution information and the structured project schedule. The system should ingest heterogeneous inputs, identify actual execution events, extract their relevant details, match them to the correct L5/L6 activity, assign confidence, and send uncertain/unmatched cases to a planner for review.

> **Target flow:** Raw field input → Relevance Detection → Event Extraction → Structured Event JSON → L5/L6 Candidate Retrieval → Semantic/Metadata Matching → Confidence → Auto-match or Human Review → Schedule/PMIS Update + Audit Trail.

0.1 WHAT WE ARE BUILD RIGHT NOW

We are **not** trying to build the entire project-management system first. We are starting with the **extractor**, because it is the first intelligence layer in the pipeline.

Extractor responsibility: convert messy human/project execution input into reliable structured events. It should answer: What work happened? What action occurred? What is the status? Was it affirmed/negated/uncertain? When did it happen? Where? Which discipline? What quantity/progress was reported? What exact text supports the extraction?

Extractor does NOT decide the final schedule_activity_id. That is a separate matching stage. Keeping the two stages separate makes the system easier to test, debug and improve.

0.2 WHY SYNTHETIC DATA IS THE STARTING POINT

The SIH26122 problem statement says live project data will not be shared and that teams should use synthetic/sample data of similar structure. Therefore, we are not blocked if Oil India sample data is unavailable.

Our synthetic dataset will be **paired**: every raw report has a known ground-truth answer. This is better for early development than scraping random construction documents because we can measure exactly whether the extractor is correct.

Public web data = realism reference. Synthetic data = primary labelled dataset. Coordinator/Oil India data = optional later validation/refinement.

1. DATA ARCHITECTURE

```
RAW INPUT ↓ RELEVANCE GATE ↓ EXTRACTOR ↓ STRUCTURED EVENTS ↓ MATCHER ↓ CONFIDENCE / REVIEW ↓ SCHEDULE UPDATE
```

The extractor should never silently guess missing information. If a date, quantity, location or other field is absent, return **null**. If the report is irrelevant, return no events. If a statement is negative, do not turn it into a completed activity.

2. DATA WE NEED

The minimum data foundation is:

• Schedule master: synthetic L5/L6 activity catalogue.

• Raw daily reports: messy natural-language execution reports.

• Gold extraction labels: the correct structured output for each raw report.

• Gold matching labels: the correct schedule activity for each extracted event.

• Terminology/alias data: abbreviations and alternative ways of describing the same work.

• Edge-case reports: ambiguous, irrelevant, negative, conflicting, duplicate and partial-progress cases.

• Later: spreadsheet, PDF/diary and voice-transcript inputs.

3. SCHEDULE DATASET

Create a synthetic infrastructure/oil-and-gas-style project with approximately **150–250 activities** initially. Include multiple disciplines and deliberately create similar activities so matching is not artificially easy.

Minimum schedule fields:

• project_id

• wbs_code

• activity_id

• activity_name

• wbs_level (L5/L6)

• discipline

• location

• planned_start

• planned_finish

• unit (optional)

• planned_quantity (optional)

• predecessor_ids (optional)

Example: activity_id = PIP-L6-0142; activity_name = “Erect 24in Line 24-XX”; discipline = Piping; location = Rack B; planned dates = 2026-09-01 to 2026-09-04.

4. RAW REPORT DATASET

Reports must contain realistic variation. The extractor should not only see clean sentences.

• Formal: “Erection of 24 inch spool at Rack B completed from 10:00 to 16:00.”

• Informal: “24in spool erection done at rack B, 10 to 4.”

• Abbreviated: “PIP - 24in spool erectn @ RB-B, 1000-1600, complete.”

• Messy/typo: “rack b 24 inch spool fitted today 10am till 4pm completed.”

• Cross-sentence: “Crew started Line 24 spool erection at 10 AM. Work was completed by 4 PM.”

• Partial: “Started erecting 24in spool. Around 5 of 12 spools completed today.”

• Start-only: “Line 24 spool erection started at 10:15 AM.”

• Finish-only: “Remaining Line 24 spool was completed at 16:00.”

• Ambiguous: “Spool erection completed in Rack B.”

• Irrelevant: “Lunch was delayed today because the canteen was crowded.”

• Mixed: “Rain stopped work for 2 hours. After that, 3 spools were erected at Rack B.”

5. REQUIRED EXTRACTOR OUTPUT

The extractor must return a stable JSON structure. This is the integration contract between the AI/core team and the backend.


```
{ "document": { "report_id": "REP-001", "source_type": "daily_report", "report_date": "2026-09-01", "discipline":
"Piping", "raw_text": "..." }, "relevance": { "is_relevant": true, "confidence": 0.98, "reason": "Contains project
execution information" }, "events": [ { "event_id": "EVT-001", "activity": { "description": "24 inch spool
erection", "action": "erection" }, "execution": { "status": "completed", "assertion": "affirmed",
"progress_percent": 100 }, "time": { "start": "2026-09-01T10:00:00", "end": "2026-09-01T16:00:00",
"time_certainty": "explicit" }, "context": { "discipline": "Piping", "location": "Rack B", "equipment": null,
"line_number": "24" }, "quantity": { "completed": null, "total": null, "unit": null }, "evidence": {
"source_text": "Piping crew completed erection of 24 inch spool at Rack B from 10 AM to 4 PM." }, "confidence": {
"overall": 0.95, "activity": 0.97, "status": 0.99, "time": 0.93 }, "warnings": [] } ] }
```


6. EXTRACTOR FIELD DEFINITIONS

• **is_relevant:** whether the input contains project execution information.

• **description:** normalized description of the physical work.

• **action:** main execution action such as erection, welding, testing, painting, installation.

• **status:** started, in_progress, completed, suspended, cancelled/not_started where supported.

• **assertion:** affirmed, negated or uncertain.

• **progress_percent:** only when explicitly reported or safely derived from explicit quantities.

• **start/end:** actual execution times; missing values remain null.

• **time_certainty:** explicit/inferred/missing.

• **discipline:** Civil, Piping, Electrical, Instrumentation, Equipment, HSE, etc.

• **location:** area, rack, unit, building or other site context.

• **line/equipment:** identifiers such as Line 24 or Pump P-101.

• **quantity:** completed/total/unit where reported.

• **evidence:** source phrase supporting the extraction.

• **warnings:** ambiguity, conflict, missing context or other flags.

7. SYNTHETIC DATA GENERATION METHOD

• Generate 150–250 synthetic L5/L6 schedule activities.

• For each activity, create 1–2 canonical execution reports.

• Generate wording variations for the same ground-truth activity.

• Generate multi-activity reports containing 2–5 events.

• Generate partial-progress and start-only/finish-only cases.

• Generate hard negatives: similar-but-wrong, unmatched and irrelevant reports.

• Generate missing-field cases where null is the correct answer.

• Freeze the gold labels independently of the extractor.

8. RECOMMENDED FIRST DATASET

If time is extremely limited: start with 100 reports.

• 40 normal reports

• 20 wording/abbreviation/noise variations

• 10 multi-event reports

• 10 partial-progress reports

• 10 ambiguous reports

• 10 irrelevant/negative reports

After the first end-to-end pipeline works, expand toward approximately **500 reports** with more edge cases.

9. CRITICAL EDGE CASES

• Different wording / synonyms

• Abbreviations and spelling errors

• Multiple activities in one report

• Partial completion

• Start-only or finish-only

• Missing dates/locations/quantities

• Two nearly identical candidate activities

• Activity not present in schedule

• Negative statement: work did not happen

• Cancelled/not-started activity

• Duplicate report

• Conflicting reports

• Field statement combines several planned activities

• Relative/overnight dates

• Weather/delay/safety/material noise mixed with progress

10. MATCHING STAGE — AFTER EXTRACTION

The matcher receives the extracted event and the schedule catalogue. It should use multiple signals: semantic similarity, discipline, location, identifiers, action, equipment/line, WBS context and quantity/unit.

Important: correct abstention is better than a confidently wrong match. If the evidence is insufficient or multiple candidates are too close, return *needs_review* instead of silently changing the schedule.

11. EVALUATION

• Relevance precision/recall

• Event detection F1

• Field accuracy

• Date/time accuracy

• Evidence coverage

• Match top-1 accuracy

• Match top-k recall

• Unsafe-match rate

The first demo should clearly report event extraction accuracy, field accuracy, top-1 match accuracy and unsafe-match rate.

12. TRAIN/VALIDATION/TEST

Use approximately 70% development, 15% validation and 15% frozen final test. Avoid placing near-identical wording variants of the same activity in both development and test, otherwise performance can be inflated by memorization.

13. RECOMMENDED PROJECT DATA STRUCTURE

```
data/ schedule/ schedule_activities.csv raw_reports/ reports.jsonl reports_excel.xlsx reports_pdf/ labels/
gold_extractions.jsonl gold_matches.jsonl terminology/ aliases.csv abbreviations.csv edge_cases/ irrelevant.jsonl
```

```
ambiguous.jsonl conflicting.jsonl negative.jsonl evaluation/ test_set.jsonl metrics.json
```

14. WHAT CLAUDE CODE SHOULD NOT BUILD YET

- Do not fine-tune a custom LLM.
- Do not build a giant RAG system.
- Do not build production OCR/ASR before text extraction works.
- Do not recreate Primavera P6.
- Do not start with delay prediction.
- Do not require thousands of real reports.
- Do not let extraction directly mutate the schedule/database.

First success condition: one messy report → one correct structured event → one correct candidate activity → confidence → accept/review decision.

15. IMMEDIATE IMPLEMENTATION PLAN

Step 1: Generate the synthetic schedule. **Step 2:** Generate the first 100 raw reports. **Step 3:** Generate gold extraction labels. **Step 4:** Implement relevance detection + extractor. **Step 5:** Implement candidate retrieval + matching. **Step 6:** Implement confidence/reviewer logic. **Step 7:** Add Excel/PDF only after the text pipeline is reliable.

Coordinator data is not a blocker. If Oil India provides sample formats later, use them to refine the synthetic generator and create a separate validation set.

16. FINAL INSTRUCTION TO THE IMPLEMENTATION AGENT

Build the system around the data contract in this document. Prioritize correctness, traceability and safe abstention over feature count. The extractor must preserve source evidence, never invent missing information, support multiple events per report, and remain independent from the schedule-matching stage. The purpose of the current phase is to establish a reliable text-based extraction benchmark before adding other input modalities.